

World Health Organization Healthcare RAG Assistant Project

Muhammad Ammar Rashid

Introduction and Problem Statement

This project involved creating a RAG assistant that answers questions about HIV service-delivery guidelines. It does this by retrieving the information from a collection of imported relevant documents, allowing for more accurate answers to be generated.

The system ingests the PDF documents and uses RecursiveCharacterTextSplitter to split the documents into smaller chunks, converts the chunks into vector embeddings, stores them in a Chroma vector database, retrieves the most relevant document chunks for the questions generated by users, and provides the retrieved context to the LLM.

The response is generated based on the information present in the most relevant chunks and it includes a citation which shows the source document and page number.

Which LLM I Chose and Why

After researching multiple Large Language Models, I chose to use the Qwen3-4B model for the following reasons:

- Firstly, Qwen3-4B can be run locally, which allows the system to function without sharing the documents or questions with an external API.
- Additionally, it is capable of following instructions that require answers to be based only on the retrieved context.
- Its parameter size (4 billion parameters) also makes it suitable for being run locally, as compared to much larger LLMs.

Which Embedding Model I Chose and Why

Initially, after going through multiple embedding models on, I chose the Qwen3-Embedding-0.6 model, since it is capable of running on CPUs and is lightweight. It is also highly efficient for local semantic search, due to its large context size.

However, after I tested it on the RAG application, I found that it was functioning at a significantly slow rate, when embedding the file content. To speed up the process, I switched to the **sentence-transformers/all-MiniLM-L6-v2** model and found that it performed the embedding generation within only about 1-2 minutes. My reasons for switching to MiniLM were the following:

- It is lightweight and suitable for local semantic search.
- The embedding generation was fast and the model is designed particularly for semantic similarity and sentence embeddings.
- It provides good semantic retrieval, while dealing with numerous document chunks.

Which Vector Database I Chose and Why

I selected ChromaDB as the vector database for the following reasons:

- It is a suitable option when working with LangChain, as they can be easily integrated.
- It is simple to set up and use and it does not require an external database server.
- It supports persistent local vector storage and provides efficient similarity search over document embeddings.

The Chunk Size I Set and the Reasoning

I used a chunk size of 1,000 characters and a chunk overlap size of 200 characters. This chunk size is effective when breaking down large documents into smaller chunks and the chunk overlap I set is also efficient, as it has a higher chance to properly maintaining context between two different chunks. Taken together, the embedding and retrieval occur properly and the chances of lost context are significantly less.

Architecture Diagram

Document Ingestion -> Embedding Generation -> ChromaDB Vector Database -> User
Question and Retrieval -> Context Consturction -> Answer Generation -> Django Backend -
> ReactFrontend

The RAG pipeline begins with document ingestion, where the PDF documents are loaded and divided into smaller chunks using RecursiveCharacterTextSplitter. These chunks are converted into numerical embeddings using sentence-transformers/all-MiniLM-L6-v2 and stoed as vectors in the ChromaDB database, along with their source and page metadata.

When the user submits a question, the system converts the question into an embedding and performs a similarity search in ChromaDB to retrieve the most relevant document chunks.

The retrieved chunks are then combined to create the context provided to Qwen3-4B, which generates an answer using only the retrieved infromation. The Django backend handles the RAG process through a REST API, receiving questions and returning the generated answer with its source citation. Finally, the React frontend provides the user interface, sending questions to Django and displaying the final answer and citation to the user.

Findings and Results

After running the RAG pipeline, I found that 15,452 records were stored in ChromaDB. I ran tests on the system by inputting questions about the HIV guidelines. Initially, I faced blockers such as the following: the generated answers were repeated, the model also returned the fallback option multiple times as the answer, the model's reasoning process was also shown in the final generated answer, etc.

To fix these issues, I updated the prompt guidelines to allow the system to generate only the desired answer and implemented response-cleaning logic to remove repetition and generate a proper answer if relevant document chunks are found and retrieved.

Consequently, I once again ran tests on the system by asking relevant questions and found that it successfully retrieves relevant document chunks based on semantic similarity. The retrieved chunks contained the source and page metadata, which allowed for citations to be generated. The Qwen3-4B successfully generated answers using the retrieved context and the React frontend was communicating properly with the Django backend.

Conclusion

The project successfully implements a complete RAG system for answering questions about HIV service-delivery guidelines. The combination of MiniLM embeddings, ChromaDB, LangChain, Qwen3-4B, the Django REST Framework and the React frontend provided a system capable of retrieving relevant information and generating answers, based on retrieved document chunks.

A major design decision I made while working on this project was switching the embedding model from Qwen3-0.6B to all-MiniLM-L6-v2, as it was significantly faster in terms of embedding generation, while effectively performing semantic retrieval.